

Sentiment Analysis of Amazon Product Review using RNN

Here, Sentiment Analysis is a technique in Natural Language Processing (NLP) which helps to identify the emotional tone behind text data. In this project, we examine that Amazon product reviews using a Recurrent Neural Network (RNN) model to sort customer feedback into various sentiment categories, rated from 1 to 5. RNNs work well with text data because they recognize patterns and relationships in the order of words within sentences.

The dataset for this project comes from the *Amazon Product Review Dataset* on Kaggle. This dataset includes customer review texts and their associated ratings. We preprocess the dataset, tokenize the text, and convert it into padded sequences before training the RNN model for sentiment classification.

Required libraries

• numpy • pandas • nltk • matplotlib • seaborn • wordcloud • textblob • re • cufflinks

```
In [1]: # importing required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import re
import seaborn as sns
```

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import tensorflow as tf
from tensorflow.keras.layers import SimpleRNN, LSTM, Dense, Dropout, Embedding, BatchNormaliza
from tensorflow.keras.models import Sequential
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.utils import pad_sequences

import warnings
warnings.filterwarnings('ignore')
```

```
C:\Users\lekha\AppData\Roaming\Python\Python311\site-packages\keras\src\export\tf2onnx_lib.py:
8: FutureWarning: In the future `np.object` will be defined as the corresponding NumPy scalar.
  if not hasattr(np, "object"):
```

Read Data

```
In [2]: ## Data Loading
data = pd.read_csv('Amazon_Product_Dataset.csv')
```

```
In [3]: data.head()
```

Out [3]:

	Review	Sentiment
0	Fast shipping but this product is very cheaply...	1
1	This case takes so long to ship and it's not e...	1
2	Good for not droids. Not good for iPhones. You...	1
3	The cable was not compatible between my macboo...	1
4	The case is nice but did not have a glow light...	1

In [4]: `data.tail()`

Out [4]:

	Review	Sentiment
24995	These cables (lightning) are far superior to m...	5
24996	This unit performs exactly as advertised. I u...	5
24997	I had the key cut at a local hardware store cu...	5
24998	I love my case I ordered from amazon. It wasn'...	5
24999	Awsome!!! I loved this little speaker. It has ...	5

In [5]:

```
# Printing shape of the dataset
print(data.shape)
```

(25000, 2)

In [6]:

```
# printing columns and rows information
print(data.info())
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 25000 entries, 0 to 24999  
Data columns (total 2 columns):  
 #   Column      Non-Null Count  Dtype  
---  ---  
 0   Review      24999 non-null  object  
 1   Sentiment    25000 non-null  int64  
dtypes: int64(1), object(1)  
memory usage: 390.8+ KB  
None
```

Exploring Data Analysis (EDA)

```
In [7]: # Looking for NULL values  
print("Null Values:\n", data.isna().sum())  
  
# dropping null values  
data = data.dropna()  
  
# again checking for NULL values  
print("Null Values after dropping:\n", data.isna().sum())
```

```
Null Values:  
Review      1  
Sentiment    0  
dtype: int64  
Null Values after dropping:  
Review      0  
Sentiment    0  
dtype: int64
```

```
In [8]: # count of unique values in Sentiment column  
data['Sentiment'].value_counts()
```

```
Out[8]: Sentiment  
1      5000  
2      5000  
3      5000  
4      5000  
5      4999  
Name: count, dtype: int64
```

Data Cleaning and Text Preprocessing

Text cleaning

```
In [9]: # downloading stopwords from nltk library  
nltk.download('stopwords')  
stop_words = set(stopwords.words('english'))  
  
# downloading punkt sentence tokenizer models  
nltk.download('punkt')  
  
# Review text Cleaning  
def clean_reviews(text):  
  
    # removing html brackets  
    regex = re.compile('<.*?>')  
    text = re.sub(regex, '', text)  
  
    # removing special characters
```

```
pattern = re.compile('[^a-zA-Z0-9\s]')
text = re.sub(pattern, '', text)

# removing numbers
pattern = re.compile('\d+')
text = re.sub(pattern, '', text)

# converting text to lower case
text = text.lower()

# Tokenization
text = word_tokenize(text)

# Stop words removal
text = [word for word in text if word not in stop_words]

return text

# applying function
data['Review'] = data['Review'].apply(clean_reviews)
```

```
[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\lekha\AppData\Roaming\nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\lekha\AppData\Roaming\nltk_data...
[nltk_data] Package punkt is already up-to-date!
```

Text Tokenization and Text Encoding

```
In [10]: tokenizer = Tokenizer()
```

```

# converting all the reviews to list to pass it as a parameter to fit_on_texts
reviews_to_list = data['Review'].tolist()
tokenizer.fit_on_texts(reviews_to_list)

# Generating text sequences
text_sequences = tokenizer.texts_to_sequences(reviews_to_list)

# setting maximum words we want in an example
max_words = 500

# Padding sequences
# - Before converting to a NumPy array, pad the sequences to ensure they all have the same length
padded_sequences = pad_sequences(text_sequences, maxlen=max_words) # Use the same max_words value

text_sequences = np.array(padded_sequences)

# one hot encoding
data = pd.get_dummies(data, columns = ['Sentiment'])

# Generating our X (input) to the model
# using pad_sequences and y (output)
X = pad_sequences(text_sequences, maxlen = max_words)
y = data[['Sentiment_1', 'Sentiment_2', 'Sentiment_3', 'Sentiment_4',
          'Sentiment_5']]
print(X.shape, y.shape)

```

(24999, 500) (24999, 5)

Train-Test Split

```
In [11]: # Train Test split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)

print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)

(19999, 500) (5000, 500) (19999, 5) (5000, 5)
```

Model Development

Build the Model

```
In [13]: # Creating a RNN model
from tensorflow.keras.layers import Input

rnn = Sequential(name="Simple_RNN")

rnn.add(Input(shape=(max_words,)))

rnn.add(Embedding(len(tokenizer.word_index)+1,
                  max_words))

rnn.add(SimpleRNN(128, activation='relu', return_sequences=True))
rnn.add(SimpleRNN(64, activation='relu'))
rnn.add(Dense(5, activation='softmax'))

rnn.summary()
```

Model: "Simple_RNN"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, 500, 500)	19,052,500
simple_rnn_2 (SimpleRNN)	(None, 500, 128)	80,512
simple_rnn_3 (SimpleRNN)	(None, 64)	12,352
dense_1 (Dense)	(None, 5)	325

Total params: 19,145,689 (73.04 MB)

Trainable params: 19,145,689 (73.04 MB)

◀ Compiling the model and Model Evaluation ▶

```
In [14]: # Compiling model
rnn.compile(
    loss="categorical_crossentropy",
    optimizer='adam',
    metrics=['accuracy']
)

# Training the model
history = rnn.fit(X_train, y_train,
                  batch_size=64,
                  epochs=2,
                  verbose=1,
                  validation_data = (X_test, y_test))
```

```
# Printing model score on test data
print("Simple_RNN Score---> ", rnn.evaluate(X_test, y_test, verbose=1))
```

```
Epoch 1/2
313/313 ————— 241s 749ms/step - accuracy: 0.3692 - loss: 1.3897 - val_accuracy:
0.4194 - val_loss: 1.2848
Epoch 2/2
313/313 ————— 223s 713ms/step - accuracy: 0.5457 - loss: 1.0790 - val_accuracy:
0.4236 - val_loss: 1.3093
157/157 ————— 16s 102ms/step - accuracy: 0.4236 - loss: 1.3093
Simple_RNN Score--> [1.309272050857544, 0.4235999882221222]
```

LSTM (Long Short Term Memory)

```
In [16]: from tensorflow.keras.layers import Input

model = Sequential(name="LSTM_Model")

model.add(Input(shape=(max_words,)))

model.add(Embedding(
    input_dim=len(tokenizer.word_index)+1,
    output_dim=128
))

# adding a LSTM Layer
model.add(LSTM(150, return_sequences=False))
BatchNormalization()
model.add(Dropout(0.5))

#adding a dense layer with activation function of relu
```

```

model.add(Dense(50, activation='relu'))
BatchNormalization()
model.add(Dropout(0.5))

# adding the final output activation with activation function of softmax
model.add(Dense(5, activation='softmax'))

# printing model summary
print(model.summary())

```

Model: "LSTM_Model"

Layer (type)	Output Shape	Param #
embedding_3 (Embedding)	(None, 500, 128)	4,877,440
lstm_1 (LSTM)	(None, 150)	167,400
dropout_2 (Dropout)	(None, 150)	0
dense_4 (Dense)	(None, 50)	7,550
dropout_3 (Dropout)	(None, 50)	0
dense_5 (Dense)	(None, 5)	255

Total params: 5,052,645 (19.27 MB)

Trainable params: 5,052,645 (19.27 MB)

Non-trainable params: 0 (0.00 B)

None

Comiling the model

```
In [17]: # Compiling the model
model.compile(
    loss="categorical_crossentropy",
    optimizer='adam',
    metrics=['accuracy']
)

# Training the GRU model
history = model.fit(X_train, y_train,
                    batch_size=64,
                    epochs=3,
                    verbose=1, validation_data=(X_test, y_test))

# Printing model score on test data
print("LSTM model Score---> ", model.evaluate(X_test, y_test, verbose=1))
```

Epoch 1/3

313/313  302s 914ms/step - accuracy: 0.3506 - loss: 1.4305 - val_accuracy: 0.4580 - val_loss: 1.2347

Epoch 2/3

313/313  283s 903ms/step - accuracy: 0.5036 - loss: 1.1523 - val_accuracy: 0.4658 - val_loss: 1.2163

Epoch 3/3

313/313  282s 902ms/step - accuracy: 0.5903 - loss: 0.9897 - val_accuracy: 0.4578 - val_loss: 1.3079

157/157  30s 190ms/step - accuracy: 0.4578 - loss: 1.3079
LSTM model Score---> [1.3078657388687134, 0.457800009059906]

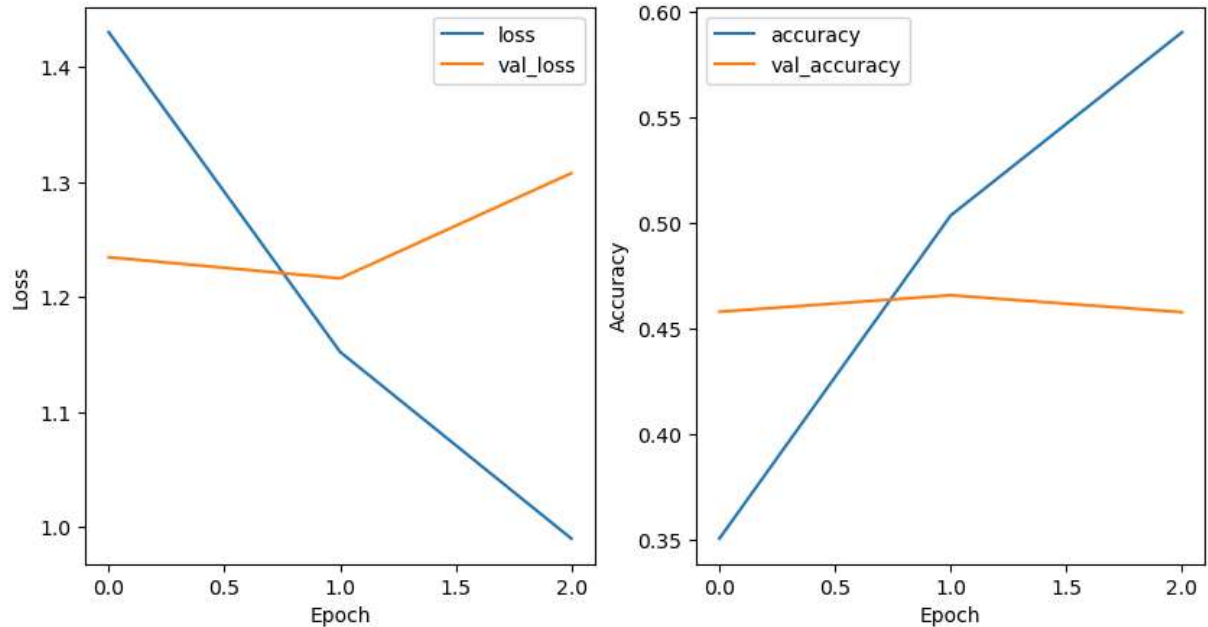
Classification

```
In [18]: # Plotting the history
metrics = history.history
plt.figure(figsize=(10, 5))

# Plotting training and validation Loss
plt.subplot(1, 2, 1)
plt.plot(history.epoch, metrics['loss'], metrics['val_loss'])
plt.legend(['loss', 'val_loss'])
plt.xlabel('Epoch')
plt.ylabel('Loss')

# Plotting training and validation accuracy
plt.subplot(1, 2, 2)
plt.plot(history.epoch, metrics['accuracy'],
          metrics['val_accuracy'])
plt.legend(['accuracy', 'val_accuracy'])
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
```

```
Out[18]: Text(0, 0.5, 'Accuracy')
```



```
In [19]: # converting y_test tht was one hot encoded to one column fo rconfusion matrix
y_true = np.argmax(y_test.values, axis=1)
y_true.shape

# Confusion matrix
y_pred = np.argmax(model.predict(X_test), axis=1)
cm = tf.math.confusion_matrix(y_true, y_pred)

# Plotting the confusion matrix
```

```
plt.figure(figsize=(10, 8))  
sns.heatmap(cm, annot=True, fmt='g')  
plt.xlabel('Predicted')  
plt.ylabel('Actual')  
plt.show()
```

157/157 ————— 32s 200ms/step





```
In [20]: from sklearn.metrics import classification_report
report = classification_report(y_true, y_pred)
print(report)
```

	precision	recall	f1-score	support
0	0.59	0.51	0.54	1021
1	0.39	0.42	0.41	1000
2	0.37	0.40	0.38	985
3	0.40	0.41	0.40	973
4	0.57	0.55	0.56	1021
accuracy			0.46	5000
macro avg	0.46	0.46	0.46	5000
weighted avg	0.47	0.46	0.46	5000

Training and Testing model

```
In [21]: def predict_review_rating(text):
text_sequences_test = np.array(tokenizer.texts_to_sequences([text]))
testing = pad_sequences(text_sequences_test, maxlen = max_words)
y_pred_test = np.argmax(model.predict(testing), axis=1)
return y_pred_test[0]+1

# Testing
rating1 = predict_review_rating('Worst product')
print("The rating according to the review is: ", rating1)
```

```
rating2 = predict_review_rating('Awesome product, I will recommend this to other users.')  
print("The rating according to the review is: ", rating2)
```

```
1/1  0s 165ms/step  
The rating according to the review is: 1  
1/1  0s 106ms/step  
The rating according to the review is: 5
```

In []:

In []:

In []: